



PERTEMUAN IV

Regression

4.1. TUJUAN PEMBELAJARAN

- A. Mahasiswa mampu memahami konsep dasar regresi
- B. Mahasiswa mengetahui prinsip kerja berbagai algoritma regresi, khususnya *Linear Regression*, *Decision Tree Regression*, dan *Random Forest Regression*
- C. Mahasiswa dapat menerapkan berbagai algoritma regresi untuk membangun model
- D. Mahasiswa mampu mengevaluasi performa model regresi menggunakan metrik RMSE, R^2 , dan koefisien korelasi (R)

4.2. DASAR TEORI

- **Regresi**

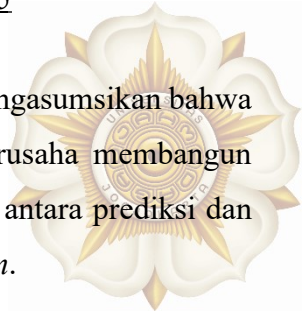
Regresi adalah metode statistik dalam *machine learning* yang digunakan untuk memodelkan hubungan antara satu atau lebih variabel independen (fitur) dengan variabel dependen (target) yang bersifat numerik atau kontinu. Tujuan utama regresi adalah untuk memahami pola hubungan antara fitur dan target, memperkirakan nilai masa depan berdasarkan data sebelumnya, dan mengidentifikasi seberapa besar pengaruh masing-masing fitur terhadap *output*.

Dalam *machine learning*, regresi termasuk dalam *supervised learning*, karena model dilatih menggunakan pasangan *input-output* sehingga bisa belajar memprediksi nilai target yang belum diketahui. Contoh penggunaan regresi antara lain yaitu prediksi harga rumah, estimasi permintaan barang, prediksi curah hujan, dan evaluasi risiko. Regresi bekerja dengan cara menemukan fungsi atau model matematis yang paling sesuai dengan pola data. Model tersebut kemudian digunakan untuk memprediksi nilai baru berdasarkan *input* baru.

- **Algoritma Regresi**

Algoritma regresi yang umum digunakan adalah *Linear Regression*, *Decision Tree Regression*, dan *Random Forest Regression*. Masing-masing algoritma memiliki karakteristik dan cara kerja yang berbeda sesuai jenis data dan pola hubungan antar variabel.

1. *Linear Regression*



Linear Regression adalah algoritma regresi paling dasar yang mengasumsikan bahwa hubungan antara fitur dan target bersifat linier. Model ini berusaha membangun sebuah garis atau *hyperplane* terbaik yang meminimalkan *error* antara prediksi dan nilai sebenarnya. Berikut merupakan persamaan *linear regression*.

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Dimana:

b_0 = intercept (nilai dasar)

b_n = koefisien regresi

x_n = nilai fitur

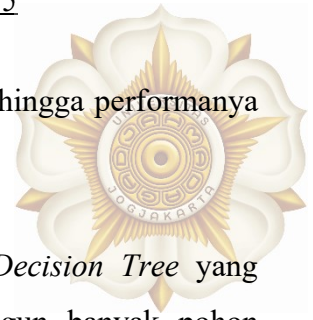
y = target/prediksi

Model *linear regression* mencari koefisien terbaik menggunakan teknik *Least Squares*, yaitu metode yang meminimalkan jumlah kuadrat *error*. Kelebihan *linear regression* yaitu sangat cepat dan efisien untuk dataset besar, mudah dipahami dan dijelaskan (*interpretable*), cocok untuk hubungan yang mendekati linear. Sedangkan kekurangannya adalah kurang cocok untuk pola non-linear dan sensitif terhadap *outlier*.

2. *Decision Tree Regression*

Decision Tree Regression merupakan salah satu algoritma regresi yang bekerja dengan membangun struktur pohon keputusan sebagai representasi model. Algoritma ini membagi data menjadi beberapa subset berdasarkan titik split yang dianggap paling optimal dalam mengurangi *error*. Setiap pembagian *node* dilakukan secara rekursif hingga mencapai *leafnode* yang berisi nilai prediksi. Pemilihan split biasanya dilakukan berdasarkan pengukuran *error* seperti *Mean Squared Error* (MSE) sehingga pemisahan data menghasilkan kelompok dengan variasi nilai target yang lebih homogen.

Decision Tree Regression sangat efektif untuk memodelkan hubungan non-linear karena struktur pohon memungkinkan model membuat batas keputusan yang kompleks. Selain itu, *decision tree* tidak memerlukan normalisasi atau *scaling* data dan dapat menangani fitur numerik maupun kategorikal. Meskipun demikian, algoritma ini memiliki kelemahan utama berupa kecenderungan *overfitting*, yaitu



ketika model terlalu menyesuaikan diri dengan data *training* sehingga performanya menurun saat diuji pada data baru.

3. *Random Forest Regression*

Random Forest Regression merupakan pengembangan dari *Decision Tree* yang menggunakan pendekatan *ensemble learning*, yaitu membangun banyak pohon keputusan dan menggabungkan prediksinya. Setiap pohon dibangun menggunakan sampel data yang dipilih secara acak melalui *bootstrap sampling*, serta subset fitur yang juga dipilih secara acak pada setiap pemisahan *node*. Proses ini menciptakan berbagai variasi pohon yang kemudian memberikan hasil prediksi lebih stabil dan tidak mudah *overfitting* dibanding *tree* tunggal.

Prediksi akhir pada *Random Forest* biasanya diperoleh dari rata-rata hasil prediksi semua pohon, sehingga model mampu menangkap pola kompleks tanpa mengorbankan akurasi. Selain itu, *Random Forest* menyediakan informasi mengenai *feature importance*, yang menunjukkan fitur mana yang paling berpengaruh dalam proses prediksi. Hal ini membuat *Random Forest* menjadi salah satu algoritma regresi yang umum digunakan, terutama pada dataset yang besar dan memiliki banyak fitur.

Meski demikian, model ini memerlukan sumber daya komputasi yang lebih besar.

• **Metrik Evaluasi Regresi**

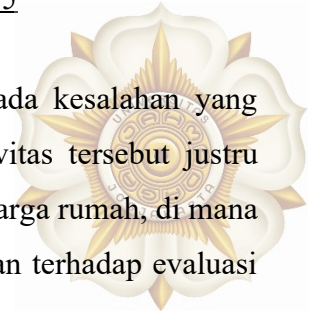
Evaluasi model regresi dilakukan menggunakan metrik khusus yang mampu mengukur kesalahan prediksi nilai numerik. Untuk menilai performa model regresi, digunakan dua metrik utama.

1. RMSE

RMSE mengukur rata-rata jarak antara nilai prediksi dan nilai sebenarnya dengan memberikan penalti lebih besar terhadap kesalahan yang besar. RMSE merupakan akar dari *Mean Squared Error*, sehingga unitnya sama dengan unit target dan lebih mudah diinterpretasikan dalam masalah nyata. Nilai RMSE yang lebih kecil menandakan bahwa model menghasilkan prediksi yang lebih akurat.

$$RMSE = \sqrt{\frac{1}{n} \sum (y_{pred} - y_{true})^2}$$

RMSE yang memiliki nilai rendah menunjukkan bahwa model mampu menghasilkan prediksi yang lebih mendekati nilai sebenarnya, sehingga tingkat akurasinya semakin



baik. Karena RMSE memberikan penalti yang lebih besar pada kesalahan yang ekstrem, metrik ini menjadi sensitif terhadap *outlier*. Sensitivitas tersebut justru menjadikan RMSE cocok digunakan pada data kontinu seperti harga rumah, di mana perbedaan nilai yang besar dapat memberikan dampak signifikan terhadap evaluasi performa model.

2. *Coefficient of Determination* (R^2),

Coefficient of Determination atau R^2 menunjukkan seberapa besar proporsi variansi target yang dapat dijelaskan oleh model. Nilai R^2 berada pada rentang 0 hingga 1, di mana nilai mendekati 1 menandakan bahwa model memiliki kemampuan prediksi yang baik dan dapat menangkap pola hubungan dengan baik. Berbeda dengan metrik klasifikasi seperti akurasi atau *precision*, RMSE dan R^2 merupakan metrik yang dirancang khusus untuk kasus regresi, karena mampu menilai kualitas prediksi numerik secara tepat.

$$0 \leq R^2 \leq 1$$

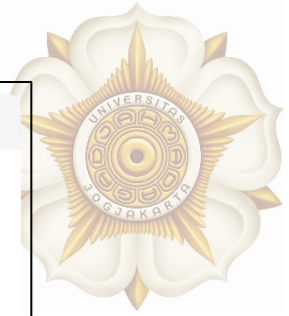
Nilai R^2 yang semakin mendekati 1 menunjukkan bahwa model mampu menjelaskan proporsi variansi target dengan lebih baik, sehingga performanya dapat dikatakan semakin sesuai dengan pola data sebenarnya. Sebaliknya, jika nilai R^2 rendah, berarti model kurang mampu menangkap hubungan antara fitur dan target. Karena sifatnya yang mengukur seberapa besar variasi data yang dapat dijelaskan oleh model, R^2 sangat berguna dalam menilai tingkat kecocokan model regresi, terutama ketika ingin memahami seberapa representatif model tersebut terhadap pola keseluruhan dalam dataset.

4.3. ALAT DAN BAHAN

- **Perangkat Keras**
 - Komputer/Laptop
- **Perangkat Lunak**
 - Python3
 - Google Colaboratory

4.4. LANGKAH PERCOBAAN

1. Impor Pustaka



```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
```

Gambar 4.4.1 Import Library

2. Load dataset

Dataset dapat diunduh pada link berikut:

<https://www.kaggle.com/datasets/shivachandel/kc-house-data>

```
df=pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/
kc_house_data.csv')
```

3. Exploratory Data Analysis (EDA)

a. Melihat 5 baris pertama dan terakhir pada dataset dengan `df.head()` dan `df.tail()`.

`df.head()`

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sqft_basement	yr_built	yr_renovated
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	1.0	0	0	...	7	1180	0	1955	0
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2.0	0	0	...	7	2170	400	1951	1991
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1.0	0	0	...	6	770	0	1933	0
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	1.0	0	0	...	7	1050	910	1965	0
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	1.0	0	0	...	8	1680	0	1987	0

5 rows × 21 columns

Gambar 4.4.2 Lima Baris Pertama Dataset

`df.tail()`

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sqft_basement	yr_built	yr_renovated
21608	263000018	20140521T000000	360000.0	3	2.50	1530	1131	3.0	0	0	...	8	1530	0	2009	
21609	6600060120	20150223T000000	400000.0	4	2.50	2310	5813	2.0	0	0	...	8	2310	0	2014	
21610	1523300141	20140623T000000	402101.0	2	0.75	1020	1350	2.0	0	0	...	7	1020	0	2009	
21611	291310100	20150116T000000	400000.0	3	2.50	1600	2388	2.0	0	0	...	8	1600	0	2004	
21612	1523300157	20141015T000000	325000.0	2	0.75	1020	1076	2.0	0	0	...	7	1020	0	2008	

5 rows × 21 columns

Gambar 4.4.3 Lima Baris Terakhir Dataset

b. Melihat deskripsi statistik dengan `df.describe()`.

```
df.describe()
```

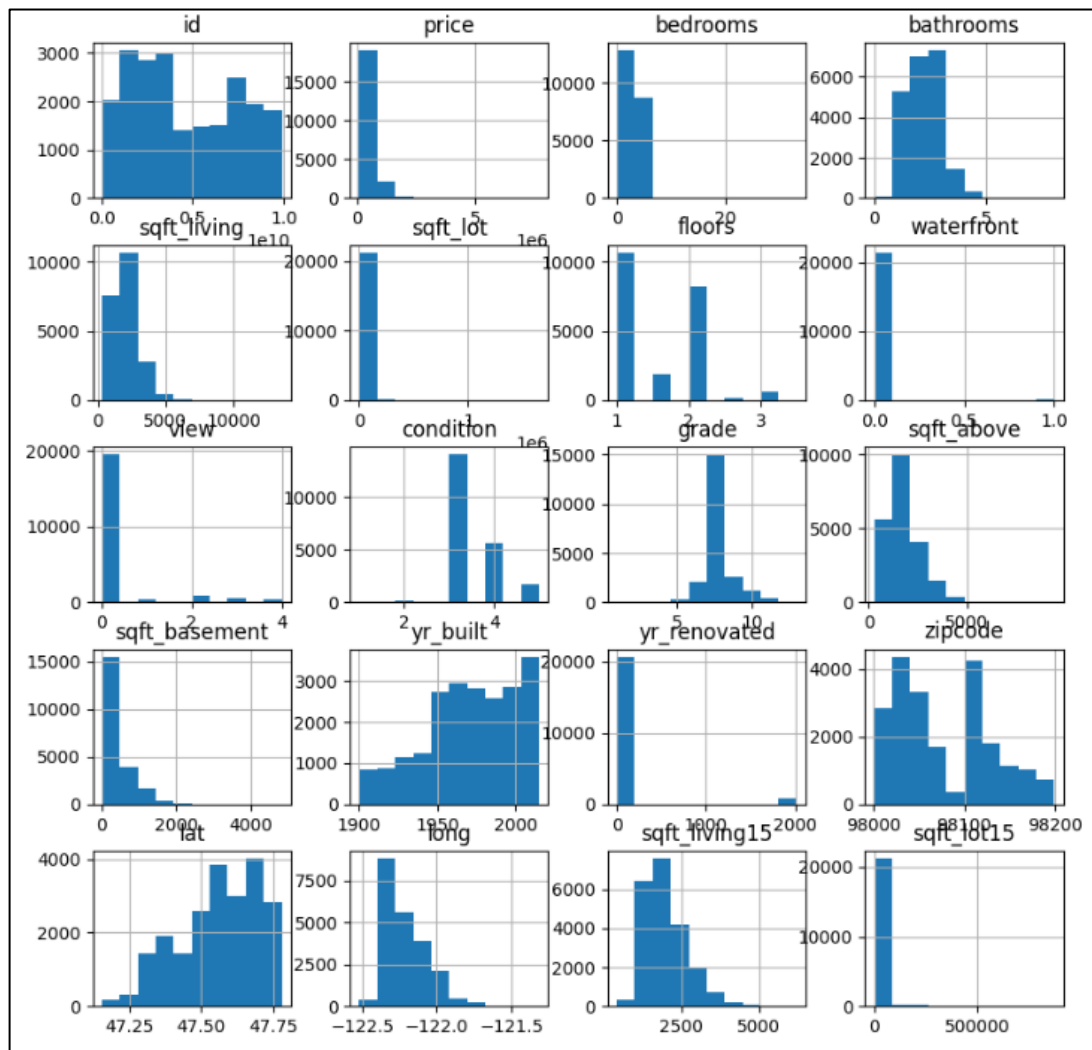
	id	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	grade	sqft_above	sqft_basement
count	2.161300e+04	2.161300e+04	21613.000000	21613.000000	21613.000000	2.161300e+04	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000
mean	4.580302e+09	5.400881e+05	3.370842	2.114757	2079.899736	1.510697e+04	1.494309	0.007542	0.234303	3.409430	7.656873	1788.390691	1788.390691
std	2.876566e+09	3.671272e+05	0.930062	0.770163	918.440897	4.142051e+04	0.539989	0.086517	0.766318	0.650743	1.175459	828.090978	828.090978
min	1.000102e+06	7.500000e+04	0.000000	0.000000	290.000000	5.200000e+02	1.000000	0.000000	0.000000	1.000000	1.000000	290.000000	290.000000
25%	2.123049e+09	3.219500e+05	3.000000	1.750000	1427.000000	5.040000e+03	1.000000	0.000000	0.000000	3.000000	7.000000	1190.000000	1190.000000
50%	3.904930e+09	4.500000e+05	3.000000	2.250000	1910.000000	7.618000e+03	1.500000	0.000000	0.000000	3.000000	7.000000	1560.000000	1560.000000
75%	7.308900e+09	6.450000e+05	4.000000	2.500000	2550.000000	1.068800e+04	2.000000	0.000000	0.000000	4.000000	8.000000	2210.000000	2210.000000
max	9.900000e+09	7.700000e+06	33.000000	8.000000	13540.000000	1.651359e+06	3.500000	1.000000	4.000000	5.000000	13.000000	9410.000000	9410.000000

Gambar 4.4.4 Hasil Deskripsi Statistik Dataset

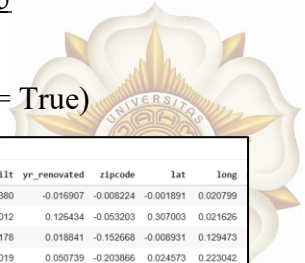
c. Melihat histogram

```
df.hist(figsize=(10,10))
plt.show()
```

Gambar 4.4.5 Kode untuk Menampilkan Histogram



Gambar 4.4.6 Histogram Seluruh Variabel Numerik



d. Mengecek korelasi atribut numeric dengan `df.corr(numeric_only = True)`

`df.corr(numeric_only = True)`

	id	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	grade	sqft_above	sqft_basement	yr_built	yr_renovated	zipcode	lat	long
id	1.000000	-0.016762	0.001286	0.005160	-0.012258	-0.132109	0.018525	-0.002721	0.011592	-0.023783	0.008130	-0.010842	-0.005151	0.021380	-0.016907	-0.008224	-0.001891	0.020799
price	-0.016762	1.000000	0.308350	0.525138	0.702035	0.089661	0.256794	0.266369	0.397293	0.036362	0.667434	0.605567	0.323816	0.054012	0.126434	-0.053203	0.307003	0.021626
bedrooms	0.001286	0.308350	1.000000	0.515884	0.576671	0.031703	0.175429	-0.005582	0.079532	0.028472	0.356967	0.477600	0.303093	0.154178	0.018841	-0.152668	-0.008931	0.129473
bathrooms	0.005160	0.525138	0.515884	1.000000	0.754665	0.087740	0.500653	0.063744	0.187737	-0.124982	0.664983	0.685342	0.283770	0.506019	0.050739	-0.203866	0.024573	0.223042
sqft_living	-0.012258	0.702035	0.576671	0.754665	1.000000	0.172826	0.353949	0.103818	0.284611	-0.058753	0.762704	0.876597	0.435043	0.318049	0.055363	-0.199430	0.052529	0.240223
sqft_lot	-0.132109	0.089661	0.031703	0.087740	0.172826	1.000000	-0.005201	0.021604	0.074710	-0.008958	0.113621	0.183512	0.015286	0.053080	0.007644	-0.129574	-0.085683	0.229521
floors	0.018525	0.256794	0.175429	0.500653	0.353949	-0.005201	1.000000	0.023698	0.029444	-0.263768	0.458183	0.523885	-0.245705	0.489319	0.006338	-0.059121	0.049614	0.125419
waterfront	-0.002721	0.266369	-0.005582	0.063744	0.103818	0.021604	0.023698	1.000000	0.401857	0.016653	0.082775	0.072075	0.080588	-0.026161	0.092885	0.030285	-0.014274	-0.041910
view	0.011592	0.397293	0.079532	0.187737	0.284611	0.074710	0.029444	0.401857	1.000000	0.045990	0.251321	0.167649	0.276947	-0.053440	0.103917	0.084827	0.006157	-0.078400
condition	-0.023783	0.036362	0.028472	-0.124982	-0.058753	-0.008958	-0.263768	0.016653	0.045990	1.000000	-0.144674	-0.158214	0.174105	-0.361417	-0.060618	0.003026	-0.014941	-0.106500
grade	0.008130	0.667434	0.356967	0.664983	0.762704	0.113621	0.458183	0.082775	0.251321	-0.144674	1.000000	0.755923	0.168392	0.446963	0.014414	-0.184862	0.114084	0.198372
sqft_above	-0.010842	0.605567	0.477600	0.685342	0.876597	0.183512	0.523885	0.072075	0.167649	-0.158214	0.755923	1.000000	-0.051943	0.423898	0.023285	-0.261190	-0.000816	0.343803
sqft_basement	-0.005151	0.323816	0.303093	0.283770	0.435043	0.015286	-0.245705	0.080588	0.276947	0.174105	0.168392	-0.051943	1.000000	-0.133124	0.071323	0.074845	0.110538	-0.144765
yr_built	0.021380	0.054012	0.154178	0.506019	0.318049	0.053080	0.489319	-0.026161	-0.053440	-0.361417	0.446963	0.423898	-0.133124	1.000000	-0.224874	-0.346869	-0.148122	0.409356
yr_renovated	-0.016907	0.126434	0.018841	0.050739	0.055363	0.007644	0.006338	0.092885	0.103917	-0.060618	0.014414	0.023285	0.071323	-0.224874	1.000000	0.064357	0.029398	-0.068372
zipcode	-0.008224	-0.053203	-0.152668	-0.203866	-0.199430	-0.129574	-0.059121	0.030285	0.084827	0.003026	-0.184862	-0.261190	0.074845	-0.346869	0.064357	1.000000	0.267048	-0.564072
lat	-0.001891	0.307003	-0.008931	0.024573	0.052529	-0.085683	0.049614	-0.014274	0.006157	-0.014941	0.114084	-0.000816	0.110538	-0.148122	0.029398	0.267048	1.000000	-0.135512
long	0.020799	0.021626	0.129473	0.223042	0.240223	0.229521	0.125419	-0.041910	-0.078400	-0.106500	0.198372	0.343803	-0.144765	0.409356	-0.068372	-0.564072	-0.135512	1.000000
sqft_living15	-0.002901	0.585379	0.391638	0.568634	0.756420	0.144608	0.279885	0.086463	0.280439	-0.092824	0.713202	0.731870	0.200355	0.326229	-0.002673	-0.279033	0.048858	0.334605
sqft_lot15	-0.136798	0.082447	0.029244	0.087175	0.183286	0.718557	-0.011269	0.030703	0.072575	-0.003406	0.119248	0.194050	0.017276	0.070958	0.007854	-0.147221	-0.086419	0.254451

Gambar 4.4.7 Korelasi Atribut Numerik

e. Mengecek missing value dengan `df.isnull().sum()`

`df.isnull().sum()`

	0
id	0
date	0
price	0
bedrooms	0
bathrooms	0
sqft_living	0
sqft_lot	0
floors	0
waterfront	0
view	0
condition	0
grade	0
sqft_above	0
sqft_basement	0
yr_built	0
yr_renovated	0
zipcode	0
lat	0
long	0
sqft_living15	0
sqft_lot15	0
dtype:	int64

Gambar 4.4.8 Pengecekan Missing Value



f. Mengecek atribut kategorikal

```
df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df['price']
cats = df_X.select_dtypes(include=['object', 'bool']).columns
print(cats)

Index([], dtype='object')
```

Gambar 4.4.9 Pengecekan Atribut Kategorikal

4. Modelling

a. Linear Regression

```
df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df['price']

X = df_X.astype(float).values
y = df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
reg = LinearRegression()
reg.fit(X_train, y_train)

print('coef of determination training ', reg.score(X_train, y_train))
print('coef of determination testing ', reg.score(X_test, y_test))

print('coefficient')
print(reg.coef_)
print('intercept')
print(reg.intercept_)

print('prediction')
y_pred = reg.predict(X_test)
print(y_pred[:10])
print('real value')
print(y_test[0:10])
```

Gambar 4.4.10 Kode untuk Membuat Model Linear Regression

- Menentukan fitur (x) dan target (y)

Proses pemodelan dimulai dengan menentukan fitur dan target yang akan digunakan oleh model. Pada kode ini, variabel `df_X` dibuat dengan menghapus kolom `id`, `date`, dan `price` dari dataset, karena kolom `price` akan menjadi target prediksi. Sementara kolom `id` dan `date` tidak bersifat informatif. Variabel `df_y` dibuat dengan kolom `price` sebagai label yang ingin diprediksi oleh model.

- Mengonversi data ke dalam bentuk numerik



Setelah fitur dan target dipisahkan, kedua variabel tersebut diubah menjadi array numerik menggunakan `.astype(float).values`.

- Membagi data menjadi data *training* dan *testing*

Tahap berikutnya adalah membagi data menjadi dua bagian, yaitu data *training* dan data *testing* menggunakan fungsi `train_test_split`. Pada kode ini, 70% data digunakan untuk melatih model dan 30% sisanya untuk menguji performa model. Parameter `random_state=42` digunakan agar pembagian data tetap konsisten setiap kali kode dijalankan.

- Melatih model

Selanjutnya, objek model *Linear Regression* dibuat dengan `reg = LinearRegression()`. Model kemudian dilatih menggunakan data *training* melalui perintah `reg.fit(X_train, y_train)`, di mana model mempelajari hubungan linier antara fitur dan target.

- Mengukur kemampuan model

Setelah tahap *training* selesai, nilai *coefficient of determination* atau R^2 dihitung baik untuk data *training* maupun *testing*.

- Melihat parameter model

Model *Linear Regression* menghasilkan dua parameter, yaitu koefisien dan *intercept*. Koefisien menunjukkan pengaruh masing-masing fitur terhadap nilai prediksi, sedangkan *intercept* merupakan nilai dasar ketika semua fitur bernilai nol. Kedua parameter ini ditampilkan dalam kode melalui `reg.coef_` dan `reg.intercept_`.

- Membuat prediksi model

Tahap selanjutnya adalah membuat prediksi pada data *testing* menggunakan `reg.predict(X_test)`. Hasil prediksi kemudian disimpan dalam variabel `y_pred`, kemudian ditampilkan untuk melihat 10 prediksi pertama. Untuk memastikan hasilnya masuk akal, nilai prediksi tersebut dibandingkan dengan 10 nilai sebenarnya dari `y_test`. Langkah ini memberikan gambaran awal mengenai akurasi model sebelum dilakukan evaluasi lebih lanjut menggunakan metrik seperti RMSE atau R^2 .



```

coef of determination training 0.6995155846436756
coef of determination testing 0.6994627057969904
coefficient
[ -3.43081477e+04  4.03129700e+04  1.12001375e+02  9.91841247e-02
  5.27154218e+03  5.43877177e+05  5.50830616e+04  2.31460673e+04
  9.49081794e+04  7.22190668e+01  3.97823082e+01 -2.59441847e+03
  2.19209734e+01 -5.56358731e+02  5.95216324e+05 -1.96904658e+05
  1.62077488e+01 -3.30430480e-01]
intercept
6641646.708095346
prediction
[ 458597.0676416  748993.75994823 1243303.75799061 1665116.95095454
 737302.05741732 283239.58524967 831732.8758231 495383.02095346
385779.81919014 474179.42285135]
real value
[ 365000.  865000. 1038000. 1490000.  711000.  211000.  790000.  680000.
 384500.  605000.]

```

Gambar 4.4.11 Hasil Prediksi Model Linear Regression

Tahap selanjutnya adalah evaluasi model dengan *metriks* RMSE dan R^2 untuk mengukur kualitas prediksi dari model regresi.

```

mse = mean_squared_error(y_test,y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test,y_pred)

print('rmse : ', rmse)
print('r2 : ', r2)

```

Gambar 4.4.12 Kode Evaluasi Model Menggunakan RMSE dan R^2

Pertama, nilai *mean squared error* (MSE) dihitung menggunakan fungsi `mean_squared_error(y_test, y_pred)`. Nilai MSE menggambarkan rata-rata kuadrat selisih antara nilai prediksi dengan nilai sebenarnya. Semakin kecil nilai MSE, semakin baik performa model. Selanjutnya, nilai *Root Mean Squared Error* (RMSE) diperoleh dengan mengambil akar kuadrat dari MSE menggunakan `np.sqrt(mse)`. RMSE memberikan interpretasi kesalahan dalam satuan yang sama dengan target, sehingga lebih mudah untuk dibandingkan.

Kemudian, performa model juga dievaluasi menggunakan R^2 (*coefficient of determination*) melalui fungsi `r2_score(y_test, y_pred)`. Nilai R^2 menunjukkan seberapa besar variansi data yang dapat dijelaskan oleh model. Nilai yang mendekati 1 berarti model memiliki kemampuan prediksi yang sangat baik, sedangkan nilai mendekati 0 menandakan model kurang mampu menjelaskan pola hubungan dalam data.

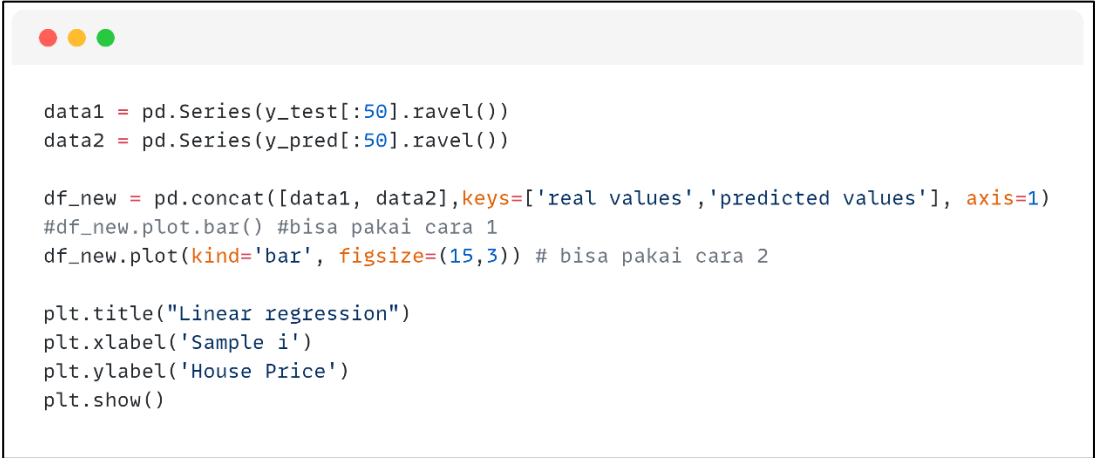


```
rmse : 208296.72772118862
r2 : 0.6994627057969904
```

Gambar 4.4.13 Hasil Evaluasi Menggunakan Metrik RMSE dan R^2

Berdasarkan hasil evaluasi, model menghasilkan nilai RMSE sebesar 208296.72, yang berarti rata-rata kesalahan prediksi model berada di kisaran angka tersebut terhadap nilai harga rumah sebenarnya. Sementara itu, nilai R^2 sebesar 0.699 menunjukkan bahwa model mampu menjelaskan sekitar 69.9% variansi dalam data harga rumah. Nilai ini mengindikasikan bahwa model *Linear Regression* memiliki kemampuan yang cukup baik dalam memodelkan hubungan antara fitur dan harga rumah, meskipun masih terdapat sekitar 30% variansi yang belum dapat dijelaskan oleh model.

Tahap selanjutnya, dilakukan visualisasi untuk membandingkan antara nilai prediksi model dan nilai sebenarnya dari dataset testing.



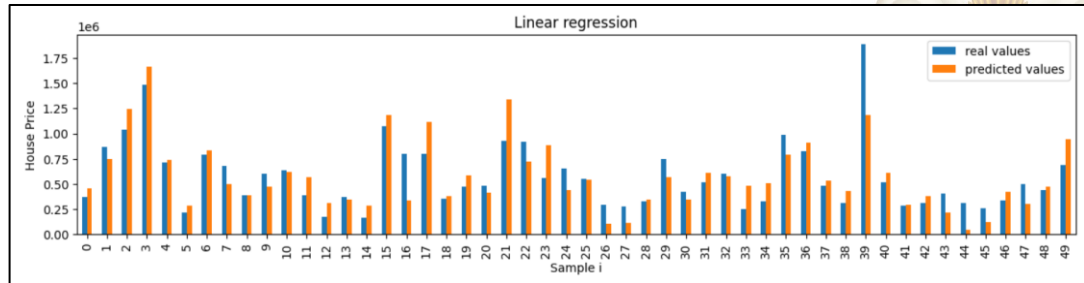
```
data1 = pd.Series(y_test[:50].ravel())
data2 = pd.Series(y_pred[:50].ravel())

df_new = pd.concat([data1, data2], keys=['real values', 'predicted values'], axis=1)
#df_new.plot.bar() #bisa pakai cara 1
df_new.plot(kind='bar', figsize=(15,3)) # bisa pakai cara 2

plt.title("Linear regression")
plt.xlabel('Sample i')
plt.ylabel('House Price')
plt.show()
```

Gambar 4.4.14 Kode Visualisasi Perbandingan Nilai Prediksi dan Nilai Sebenarnya

- Pertama, diambil 50 sampel awal dari `y_test` dan `y_pred`, kemudian masing-masing diubah menjadi objek `Series`.
- Kedua `Series` ini digabungkan menjadi sebuah `DataFrame` bernama `df_new`, dengan dua kolom yang diberi label *real values* dan *predicted values*.
- Setelah data digabung, grafik batang (*bar chart*) dibuat menggunakan perintah `df_new.plot(kind='bar', figsize=(15,3))`. Visualisasi ini berfungsi sebagai alat untuk mengevaluasi apakah pola prediksi model mendekati pola sebenarnya.



Gambar 4.4.15 Grafik Perbandingan Nilai Prediksi dan Nilai Sebenarnya

b. Decision Tree Regression

```

df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df['price']

X = df_X.astype(float).values
y = df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
dt = DecisionTreeRegressor(max_depth=10)
dt.fit(X_train, y_train)

print('coef of determination training ', dt.score(X_train, y_train))
print('coef of determination testing ', dt.score(X_test, y_test))

print('prediction')
y_pred = dt.predict(X_test)
print(y_pred[:10])
print('real value')
print(y_test[:10])

```

Gambar 4.4.16 Kode untuk Membuat Model Decision Tree Regression

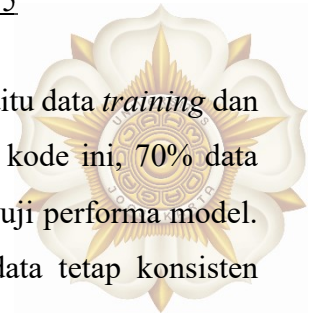
- Menentukan fitur (x) dan target (y)

Proses pemodelan dimulai dengan menentukan fitur dan target yang akan digunakan oleh model. Pada kode ini, variabel `df_X` dibuat dengan menghapus kolom `id`, `date`, dan `price` dari dataset, karena kolom `price` akan menjadi target prediksi. Sementara kolom `id` dan `date` tidak bersifat informatif. Variabel `df_y` dibuat dengan kolom `price` sebagai label yang ingin diprediksi oleh model.

- Mengonversi data ke dalam bentuk numerik

Setelah fitur dan target dipisahkan, kedua variabel tersebut diubah menjadi array numerik menggunakan `.astype(float).values`.

- Membagi data menjadi data *training* dan *testing*



Tahap berikutnya adalah membagi data menjadi dua bagian, yaitu data *training* dan data *testing* menggunakan fungsi `train_test_split`. Pada kode ini, 70% data digunakan untuk melatih model dan 30% sisanya untuk menguji performa model. Parameter `random_state=42` digunakan agar pembagian data tetap konsisten setiap kali kode dijalankan.

- Melatih model

Model *Decision Tree Regression* dibuat dengan objek `DecisionTreeRegressor(max_depth=10)` yang berfungsi sebagai representasi pohon keputusan untuk tugas regresi. Parameter `max_depth=10` digunakan untuk membatasi kedalaman maksimum pohon sehingga model tidak terlalu kompleks dan dapat mengurangi risiko *overfitting*. Setelah objek model dibuat, tahap *training* dilakukan dengan memanggil `dt.fit(X_train, y_train)` sehingga model mempelajari pola hubungan antara fitur pada data latih dan nilai targetnya.

- Mengukur kemampuan model

Setelah tahap *training* selesai, nilai *coefficient of determination* atau R^2 dihitung baik untuk data *training* maupun *testing*.

- Membuat prediksi model

Setelah model selesai dilatih menggunakan data *training*, langkah berikutnya adalah membuat prediksi terhadap data yang belum pernah dilihat model, yaitu data *testing*. Proses ini dilakukan menggunakan perintah `dt.predict(X_test)`, yang menghasilkan serangkaian nilai hasil prediksi dan disimpan dalam variabel `y_pred`.

```
coef of determination training  0.9185274272472612
coef of determination testing  0.7689346283378805
prediction
[ 372512.5      837693.06862745 1087247.96153846 1984617.64705882
 656314.08658009 237079.87179487  849106.21276596 517991.26363636
 366270.875    519039.40166667]
real value
[ 365000.  865000. 1038000. 1490000.  711000.  211000.  790000.  680000.
 384500.  605000.]
```

Gambar 4.4.17 Hasil Prediksi Model Decision Tree Regression

Selanjutnya dilakukan evaluasi model menggunakan metrik seperti RMSE dan R^2 . Berikut hasil evaluasi model menggunakan algoritma *Decision Tree Regression*.



```
mse = mean_squared_error(y_test,y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test,y_pred)

print('rmse : ', rmse)
print('r2 : ', r2)
```

Gambar 4.4.18 Kode Evaluasi Model Menggunakan RMSE dan R2

```
rmse : 182642.01673989513
r2 : 0.7689346283378805
```

Gambar 4.4.19 Hasil Evaluasi Model

Tahap selanjutnya, dilakukan visualisasi untuk membandingkan antara nilai prediksi model dan nilai sebenarnya dari dataset *testing*.

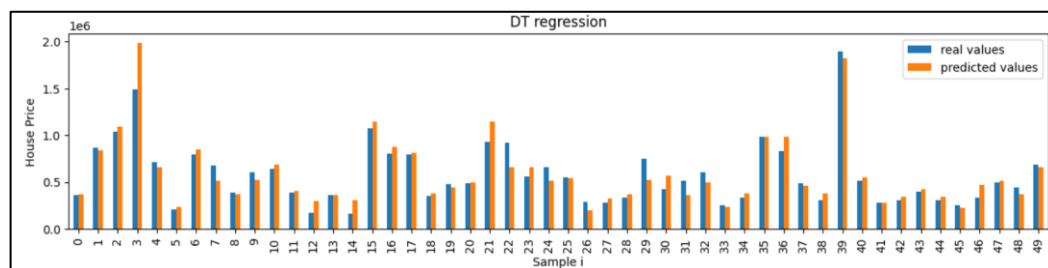
```
data1 = pd.Series(y_test[:50].ravel())
data2 = pd.Series(y_pred[:50].ravel())

df_new = pd.concat([data1, data2],keys=['real values','predicted values'], axis=1)
#df_new.plot.bar() #bisa pakai cara 1
df_new.plot(kind='bar', figsize=(15,3)) # bisa pakai cara 2

plt.title("DT regression")
plt.xlabel('Sample i')
plt.ylabel('House Price')
plt.show()
```

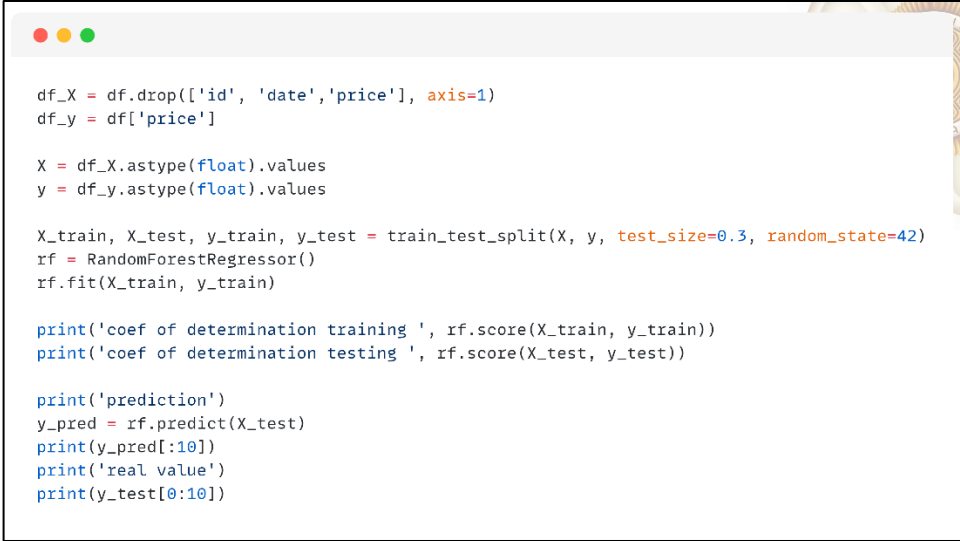
Gambar 4.4.20 Kode Visualisasi Perbandingan Nilai Prediksi dan Nilai Sebenarnya

Berikut hasil perbandingan antara nilai harga rumah sebenarnya dan nilai hasil prediksi model *Decision Tree Regression* untuk 50 sampel pertama dari data testing.



Gambar 4.4.21 Grafik Perbandingan Nilai Prediksi dan Nilai Sebenarnya

c. Random Forest Regression



```
df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df['price']

X = df_X.astype(float).values
y = df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
rf = RandomForestRegressor()
rf.fit(X_train, y_train)

print('coef of determination training ', rf.score(X_train, y_train))
print('coef of determination testing ', rf.score(X_test, y_test))

print('prediction')
y_pred = rf.predict(X_test)
print(y_pred[:10])
print('real value')
print(y_test[:10])
```

Gambar 4.4.22 Kode untuk Membuat Model Random Forest Regression

- Menentukan fitur (x) dan target (y)

Proses pemodelan dimulai dengan menentukan fitur dan target yang akan digunakan oleh model. Pada kode ini, variabel `df_X` dibuat dengan menghapus kolom `id`, `date`, dan `price` dari dataset, karena kolom `price` akan menjadi target prediksi. Sementara kolom `id` dan `date` tidak bersifat informatif. Variabel `df_y` dibuat dengan kolom `price` sebagai label yang ingin diprediksi oleh model.

- Mengonversi data ke dalam bentuk numerik

Setelah fitur dan target dipisahkan, kedua variabel tersebut diubah menjadi array numerik menggunakan `.astype(float).values`.

- Membagi data menjadi data *training* dan *testing*

Tahap berikutnya adalah membagi data menjadi dua bagian, yaitu data *training* dan data *testing* menggunakan fungsi `train_test_split`. Pada kode ini, 70% data digunakan untuk melatih model dan 30% sisanya untuk menguji performa model. Parameter `random_state=42` digunakan agar pembagian data tetap konsisten setiap kali kode dijalankan.

- Melatih model

Tahap pemodelan *Random Forest Regression* dimulai dengan membentuk objek model melalui `RandomForestRegressor()`, yang secara *default* akan membuat sekumpulan pohon keputusan sebagai dasar prediksi. Model ini kemudian dilatih menggunakan data *training* dengan memanggil `rf.fit(X_train, y_train)`,



sehingga setiap pohon dalam *ensemble* mempelajari pola hubungan antara fitur dan harga rumah dari subset data yang berbeda.

- Mengukur kemampuan model

Setelah tahap *training* selesai, nilai *coefficient of determination* atau R^2 dihitung baik untuk data *training* maupun *testing*.

- Membuat prediksi model

Tahap berikutnya adalah membuat prediksi menggunakan `rf.predict(X_test)`, yang menghasilkan nilai estimasi harga rumah dengan data testing.

```
coef of determination training 0.9822001440062388
coef of determination testing 0.8596089494795431
prediction
[ 379509.      885196.      1130435.43333333 2179107.
  703632.1    249551.38    838150.5      630879.5
  404713.92    541167.67      ]
real value
[ 365000.  865000. 1038000. 1490000.  711000.  211000.  790000.  680000.
 384500.  605000.]
```

Gambar 4.4.23 Hasil Prediksi Model Random Forest Regression

Selanjutnya dilakukan evaluasi model menggunakan metrik seperti RMSE dan R^2 . Berikut hasil evaluasi model menggunakan algoritma *Decision Tree Regression*.

```
mse = mean_squared_error(y_test,y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test,y_pred)

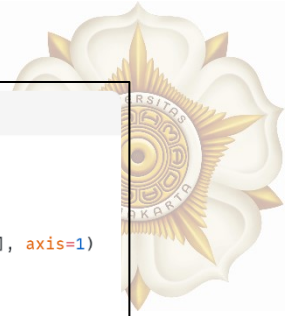
print('rmse : ', rmse)
print('r2 : ', r2)
```

Gambar 4.4.24 Kode Evaluasi Model Menggunakan RMSE dan R^2

```
rmse : 142364.91502742676
r2 : 0.8596089494795431
```

Gambar 4.4.25 Hasil Evaluasi Model

Tahap selanjutnya, dilakukan visualisasi untuk membandingkan antara nilai prediksi model dan nilai sebenarnya dari dataset *testing*.



```

data1 = pd.Series(y_test[:50].ravel())
data2 = pd.Series(y_pred[:50].ravel())

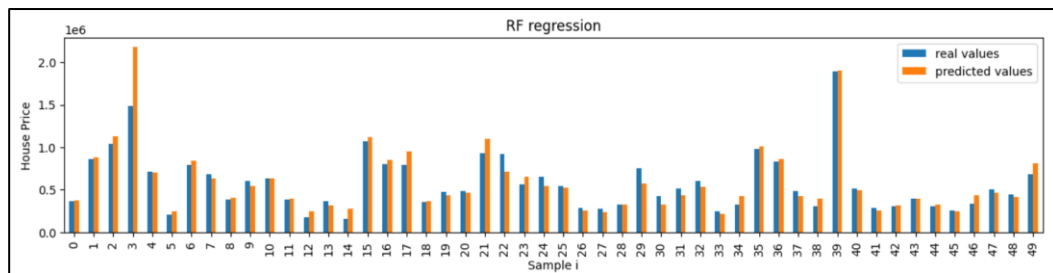
df_new = pd.concat([data1, data2], keys=['real values', 'predicted values'], axis=1)
#df_new.plot.bar() #bisa pakai cara 1
df_new.plot(kind='bar', figsize=(15,3)) # bisa pakai cara 2

plt.title("RF regression")
plt.xlabel('Sample i')
plt.ylabel('House Price')
plt.show()

```

Gambar 4.4.26 Kode Visualisasi Perbandingan Nilai Prediksi dan Nilai Sebenarnya

Berikut hasil perbandingan antara nilai harga rumah sebenarnya dan nilai hasil prediksi model *Random Forest Regression* untuk 50 sampel pertama dari data testing.



Gambar 4.4.27 Grafik Perbandingan Nilai Prediksi dan Nilai Sebenarnya

4.5. TUGAS & ANALISIS

Dataset: <https://www.kaggle.com/datasets/hellbuoy/car-price-prediction/data>

1. Jelaskan apa tujuan penggunaan dataset ini dan definisikan atribut yang menjadi input dan *output*-nya!
2. Sebutkan variabel yang paling mempengaruhi *output* atau mempunyai nilai korelasi tinggi!
3. Silahkan membuat model regresi dengan *Linear Regression*, *Decision Tree Regression*, *Random Forest Regression* dari dataset diatas!
4. Buatlah tabel yang menjelaskan performa dari model *machine learning* untuk kasus dataset diatas! Kolom pertama "model", kolom selanjutnya "RMSE, R2, R".
5. Jelaskan dari hasil eksperimen di atas, model mana yang paling baik? Jelaskan alasan anda!

4.6. REFERENSI

Han, J., Pei, J., & Tong, H. (2022). Data Mining: Concepts and Techniques (4th ed.). Morgan Kaufmann/Elsevier.

